TRABALHO PRÁTICO

AES-128 entre dois ESP32

Criptografia, comunicação UART e recuperação da mensagem.

Uma mensagem.
Duas placas.
O mesmo texto no final.



DADOS REAIS DA DUPLA

Augusto · 2023178440013
Gustavo · 202127844251

ENTENDA O SISTEMA

O que queremos mostrar?

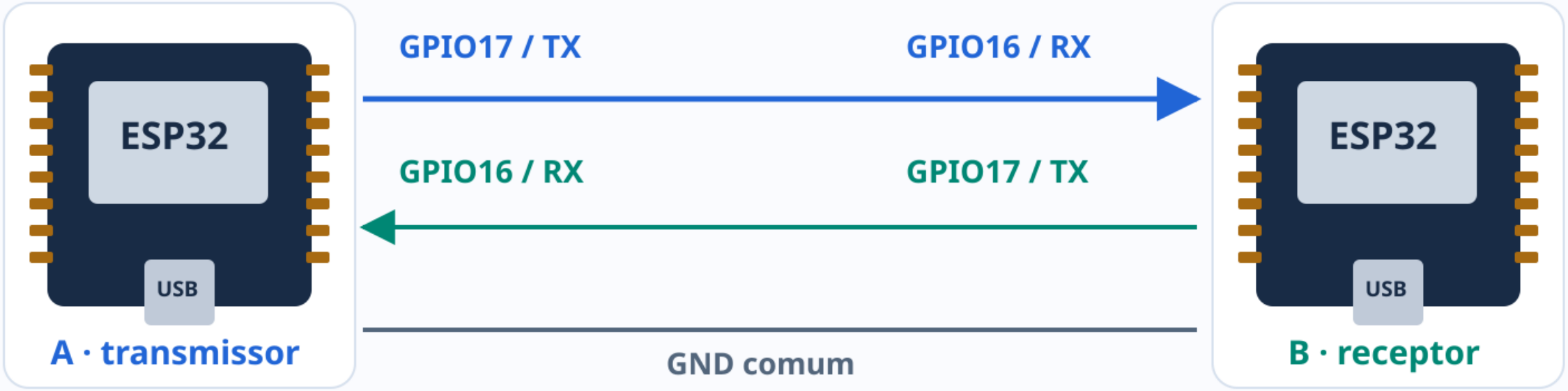
A mensagem muda de aparência durante o transporte e volta ao texto original.



O teste final compara o texto original com o texto recuperado, byte por byte.

Dois caminhos: UART para dados, USB para observar

TX é a saída de dados. RX é a entrada. GND dá uma referência elétrica comum.



UART2: 115200 bps, 8N1. USB de cada placa: um monitor serial.

ENTENDA O SISTEMA

Um texto também é uma sequência de números

Byte = grupo de 8 bits. Hexadecimal = uma forma curta de escrever esses números.

CARACTERE	=	BYTE EM HEX	=	BITS
O símbolo que lemos: 7	=	O código ASCII de “7”: 0x37	=	O mesmo byte: 00110111

AES-128 usa uma chave de 128 bits e trabalha com blocos de 16 bytes.

DADOS DA DUPLA

A chave nasce das matrículas

Pegamos os últimos 8 dígitos de A e os últimos 8 de B. Depois juntamos A + B.

A · Augusto

2023178440013 → 78440013

B · Gustavo

202127844251 → 27844251

CHAVE ASCII · 16 BYTES

7844001327844251

Regra do trabalho: dados previsíveis. Em um produto, a chave deve ser secreta e imprevisível.

PREPARAR A MENSAGEM

O AES precisa de blocos completos

Nossa mensagem tem 30 bytes. O PKCS#7 acrescenta 02 02 para chegar a 32.

A:2023178440013;B:202127844251

BLOCO 1	41	3a	32	30	32	33	31	37	38	34	34	30	30	31	33	3b
BLOCO 2	42	3a	32	30	32	31	32	37	38	34	34	32	35	31	02	02

Se a mensagem já for múltipla de 16, entram 16 bytes de valor 0x10.

CÓDIGO REAL

No código: completar e cifrar

A preparação do texto e a cifra são etapas diferentes.

pkcs7_ecb.c · dentro de pkcs7_preencher()

```
const uint8_t falta = (uint8_t)(total - tamanho);  // 1..16
if (tamanho) memmove(saida, msg, tamanho);
memset(saida + tamanho, falta, falta);
```

pkcs7_ecb.c · dentro de ecb_cifrar()

```
for (size_t b = 0; b < blocos; ++b)
  aes128_cifrar_bloco(ctx, in + 16 * b, out + 16 * b);
```

Cada chamada trabalha com 16 bytes. A mesma chave é usada nos dois lados.

O AES transforma um bloco em 10 rodadas

A cada rodada, substituímos, deslocamos, misturamos e combinamos com a chave.

ANTES DAS RODADAS

AddRoundKey com a chave 0

SubBytes

Troca cada byte

ShiftRows

Desloca as linhas

MixColumns

Mistura as colunas

AddRoundKey

Combina com a chave

Rodadas 1 a 9: as quatro etapas. Rodada 10: sem MixColumns.

TRANSFORMAÇÃO 1

SubBytes: trocar cada byte por outro

A tabela S-box define a substituição. Isso dificulta relações simples entre entrada e saída.



Exemplo do primeiro bloco da dupla.

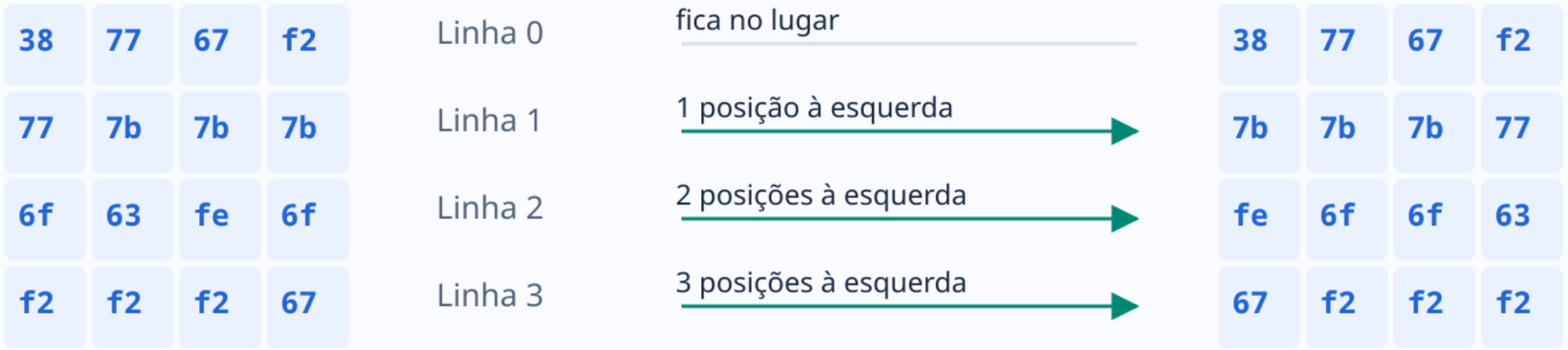
aes128.c · código real

```
void aes_sub_bytes(uint8_t s[16]) {
  for (int i = 0; i < 16; ++i) s[i] = AES_SBOX[s[i]];
}
```

TRANSFORMAÇÃO 2

ShiftRows: cada linha anda um pouco

Linha 0 fica. As outras giram 1, 2 e 3 posições para a esquerda.



TRANSFORMAÇÃO 3

MixColumns: uma coluna se mistura por dentro

Cada byte de saída depende dos quatro bytes daquela coluna.



aes128.c · multiplicação por 02

```
uint8_t aes_xtime(uint8_t a) {
  return (uint8_t)((a << 1) ^ ((a & 0x80) ? 0x1b : 0x00));
}
```

TRANSFORMAÇÃO 4

AddRoundKey: a chave entra na transformação

Usamos XOR: bits iguais dão 0; bits diferentes dão 1.

41	XOR	37	=	76
byte da mensagem		byte da chave		novo byte

aes128.c · código real

```
void aes_add_round_key(uint8_t s[16], const uint8_t chave_rodada[16]) {
  for (int i = 0; i < 16; ++i) s[i] ^= chave_rodada[i];
}
```

EXPANSÃO DE CHAVE

Uma chave inicial gera as chaves das rodadas

AES-128 usa 11 chaves de 16 bytes: a inicial e uma para cada rodada.

RotWord
Gira bytes

→

SubWord
Aplica S-box

→

Rcon
Inclui constante

→

XOR
Deriva palavras

CHAVE DA RODADA 0

`37383434303031333237383434323531`

CHAVE DA RODADA 1

`15aef32c259ec21f17a9fa2b239bcf1a`

CÓDIGO REAL

O ciclo completo aparece neste trecho

A última rodada pula MixColumns, como exige o AES.

aes128.c · dentro de aes128_cifrar_bloco()

```
  aes_add_round_key(s, ctx->chaves_rodada[0]);
  for (int r = 1; r <= AES128_RODADAS; ++r) {
    aes_sub_bytes(s);
    aes_shift_rows(s);
    if (r != AES128_RODADAS) aes_mix_columns(s);
    aes_add_round_key(s, ctx->chaves_rodada[r]);
  }
```

Para voltar ao texto: etapas inversas e chaves usadas da rodada 10 até a 0.

Como B sabe onde a mensagem começa e termina?

O quadro informa o início e a quantidade de blocos cifrados.

AA

início
1 byte

02

2 blocos
1 byte

ciphertext 1

dados cifrados
16 bytes

ciphertext 2

dados cifrados
16 bytes

transmissor.cpp · bytes brutos na UART2

```
const size_t escritos = enlace.write(quadro, tamanho_quadro);
enlace.flush();
```

CÓDIGO REAL

Na placa B, o caminho acontece ao contrário

Receber → decifrar → conferir o preenchimento → recuperar a mensagem.

receptor.cpp · dentro de processar_quadro()

```
ecb_decifrar(&g_ctx, rx.dados, claro, rx.n);
log_blocos("DECIFRADO_COM_PKCS7", claro, tamanho);
size_t tamanho_msg = 0;
const pkcs7_resultado r = pkcs7_validar(claro, tamanho, &tamanho_msg);
```

PADDING VÁLIDO

Remove o preenchimento e mostra o texto.

PADDING INVÁLIDO

Descarta a mensagem e registra o motivo.

DEMONSTRAÇÃO

O que a turma verá nos dois monitores

Trechos reais dos logs de 19/10/2026, com as matrículas da dupla.

MONITOR A · transmite

```
[A] MENSAGEM_ORIGINAL tamanho=30
A:2023178440013;B:202127844251

[A] CIFRADO_ENVIADO bloco 1:
2e3e460ec295383cec6a018765ff0531
[A] CIFRADO_ENVIADO bloco 2:
30b51beef909b9d823300de92a5e9fb3
```

MONITOR B · recebe

```
[B] CIFRADO_RECEBIDO bloco 1:
2e3e460ec295383cec6a018765ff0531
[B] CIFRADO_RECEBIDO bloco 2:
30b51beef909b9d823300de92a5e9fb3

[B] MENSAGEM_RECUPERADA tamanho=30
A:2023178440013;B:202127844251
```

Registro real de B: COMPARACAO igual_a_mensagem_da_dupla=SIM

Demonstração ao vivo: três passos

Usar os dois monitores USB a 115200 bps, com envio de linha LF.

1 · PREPARAR

A e B: CHAVERESET
A: MSGRESET
B: aguardar PRONTO

2 · ENVIAR

A: ENVIAR

ou apertar BOOT
após o boot normal.

3 · CONFERIR

Mesmos blocos cifrados.

Texto recuperado igual ao original.

Alteração na arguição: MSG <novo texto>. Para voltar: MSGRESET e ENVIAR.

EVIDÊNCIAS REAIS

Como sabemos que o resultado está correto?

Conferimos valores conhecidos antes de usar a chave da dupla.

71 / 71

Verificações no computador

11 / 11

Autoteste na placa A

11 / 11

Autoteste na placa B

FIPS-197 C.1 · ciphertext esperado e conferido

`69c4e0d86a7b0430d8cdb78070b4c55a`

Mil pacotes AES, latência AES e envio AES B→A: sem validação documentada.

RESULTADOS DA DUPLA

Um único bit muda muitos bits do resultado

Avalanche: contar as diferenças entre os dois textos cifrados.

MUDANÇA NA CHAVE



142 / 256 bits · 55,47%

MUDANÇA NA MENSAGEM · SOMENTE BLOCO 1



69 / 128 bits · 53,91%

O bloco 2 ficou igual no segundo teste: em ECB, os blocos são independentes.

Por que o ECB revela padrões?

Com a mesma chave, blocos iguais produzem blocos cifrados iguais.

BLOCO 1	`LIGAR LED VERDE.`	→	`e0c22a6f … 42203dec`
BLOCO 2	`DESLIGAR LED`	→	`2b4d065e … 4cd5070`
BLOCO 3	`LIGAR LED VERDE.`	→	`e0c22a6f … 42203dec`

Um observador consegue reconhecer repetições mesmo sem conhecer o texto.

Cifrar não garante que ninguém alterou os dados

ECB não autentica: um atacante pode repetir ou trocar blocos.

NO TRABALHO

ECB + PKCS#7

Ajuda a estudar a cifra e enxergar blocos repetidos.

EM UM PRODUTO

Um modo autenticado, como GCM, com chave secreta e nonce único por chave.

Padding inválido pode revelar um erro. Padding válido não comprova integridade.

CONVERSA COM A TURMA

O que conseguimos demonstrar

O código faz as transformações e as placas recuperam a mensagem da dupla.

01 **A transforma o texto em bytes cifrados.**

02 **A UART transporta o quadro binário.**

03 **B recupera exatamente a mensagem original.**

Pergunta para a turma: se dois blocos forem iguais, o que o ECB deixa aparecer?

Apoio: os cinco estados do primeiro bloco

Valores reais da rodada 1. Hexadecimal na ordem dos bytes de entrada/saída.

1 · AddRoundKey inicial	`76020604020300040a030c040403060a`
2 · SubBytes	`38776ff2777b63f2677bfef2f27b6f67`
3 · ShiftRows	`387bfe67777b6ff2677b6ff2f27763f2`
4 · MixColumns	`64b00d03fec2df72ded2cf42f74b4ee6`
5 · AddRoundKey · chave 1	`711efe2fdb5c1d6dc97b3569d4d081fc`

Apoio: a decifragem desfaz o caminho

A placa B usa as operações inversas com as chaves em ordem contrária.

aes128.c · dentro de aes128_decifrar_bloco()

```
  aes_add_round_key(s, ctx->chaves_rodada[AES128_RODADAS]);
  for (int r = AES128_RODADAS - 1; r >= 0; --r) {
    aes_inv_shift_rows(s);
    aes_inv_sub_bytes(s);
    aes_add_round_key(s, ctx->chaves_rodada[r]);
    if (r != 0) aes_inv_mix_columns(s);
  }
```

Apoio: encontre a resposta que não chegou

Atividade complementar com os logs históricos do diagnóstico sem criptografia.

B · RESPONDEU

1.000 PINGs recebidos
1.000 PONGs registrados no envio

A · CONTOU

999 PONGs recebidos
Faltou a sequência 1

Pergunta: o enlace funcionou? E passou no teste de zero perdas?

O enlace foi demonstrado nos dois sentidos. O resumo estrito terminou FAIL.

Fontes e preparação final

O material usa os arquivos locais do trabalho e os logs já existentes.

FONTES

Enunciado: AES_Trabalho.pdf

Código: Codigo/src/

Evidências: aes-dupla e aes-ensaio, 19/10/2026.

ANTES DA AULA

Revisar o código do colega.

Concluir o README de execução.

Ensaiar os dois monitores e a alteração pedida pelo professor.